

© Joshua M. Willman 2020

J. M. Willman, *Beginning PyQt*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-5857-6_3

3. Adding More Functionality to Interfaces

Joshua M. Willman¹

(1) Hampton, VA, USA

In Chapter 2, we took a look at how to get started in PyQt, set up the main window, and learned how to create and arrange multiple QLabel widgets to create a simple application. However, none of what we did was very interactive. What good is a user interface if all you can do is stare at it?

You're in luck because this chapter is all about setting you on the path to making interfaces that are more interactive and responsive. We will take a look at some new fundamental widgets that will help us to build our next project, a functional login GUI. To make things clearer and easier for you to follow along, the login GUI will be divided into two parts, the actual login interface and a new user registration window.

Before we get started, let's take a look at the new widgets and useful concepts that will be covered in this chapter.

1. Learn about new kinds of widgets and classes, including
 - a. QPushButton – One of the most common widgets for giving our computer simple commands
 - b. QLineEdit – Which gives the user fields to input information
 - c. QCheckBox – Which can act as a binary switch
 - d. QMessageBox – Useful for displaying alert or information dialog boxes
2. Find out about event handling with Signals and Slots in PyQt.
3. Understand the differences between windows and dialog boxes when creating UIs.

Project 3.1 – Login GUI

While it might not seem like much, the login GUI, or the login screen, is probably one of the most common interfaces you interact with on a regular basis. Signing into your computer, your online bank account, e-mail, or social media accounts, logging into your phone, or signing up for some new app, the login GUI is everywhere.

The login GUI can appear to be quite a simple user interface. However, it is actually very complex for a number of reasons. First of all, it acts as the interface that allows us to access our own personal data. You want to create a GUI that clearly labels its widgets, differentiates between where to sign in and where to register a new account, and helps users to better navigate through potential errors, such as if caps lock is on or if the user-

name is incorrect. Secondly, the appearance of the login GUI and methods in which we log in to our devices have changed dramatically over the years, allowing users to log in using Touch ID or their social media accounts. This means that there is no single design that will work for every platform.

For this project, we are going to focus on creating a simple login UI that

- Allows the user to enter their username and password and calls a function to check if their information matches one that is stored in a text file
- Displays appropriate messages depending upon whether login is successful, if an error has occurred, or if we simply want to close the window
- Displays or hides the password by clicking a checkbox
- Allows the user to create a new account by clicking a “sign up” button that will open a new window

Note There are two projects in this chapter, the `login GUI` and the `create new user GUI`. They are actually one entire project that has been separated into two parts to make it easier for you to follow along, or for those who only need one part of the project and not the other.

After following along with this chapter, you will be able to make a login GUI like the one seen in Figure 3-1.



Figure 3-1 Simple login GUI

Design the Login GUI

While the look and layout of the login GUI may change between platforms, they generally have a few key components that are common throughout, such as

- Username and password entry fields
- Checkboxes that may remember the user’s login information or reveal the password
- Buttons that users can click to log in or even register for a new account

For this project, we will focus on trying to implement most of those features. (The “remember me” checkbox that is common in a lot of login

GUIs is beyond the scope of this chapter as it involves using cookies or working with PyQt's QSettings class.)

The layout for our login GUI can be seen in Figure 3-2. For this project, we will need to create a few QLabels to help users understand the purpose of this application and to give titles to our username and password entry fields.

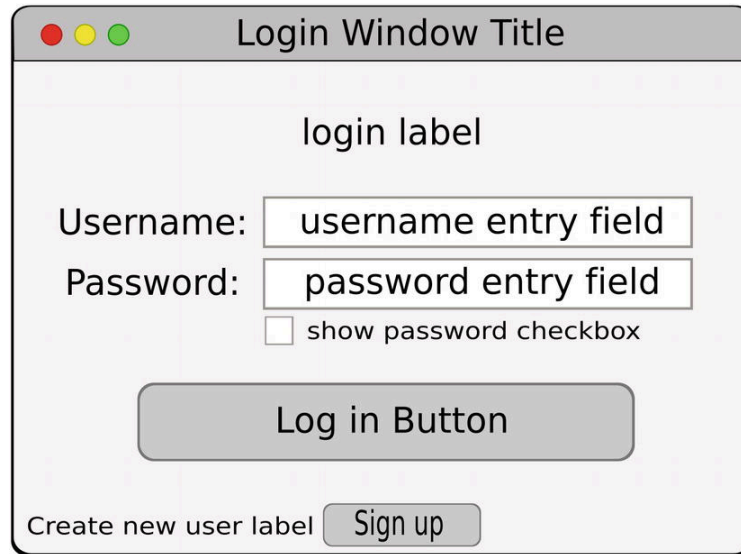


Figure 3-2 Layout for login GUI

For the areas where users will enter their information, we create two separate QLineEdit widgets. Under the password line edit widget, there is a checkbox that the user can check if they want to view or hide the password they entered.

There are two QPushButton, one that the user can click to log in and the other to register a new account. When the user clicks the login button, we will create a function that is called to check if the user exists. If the user information is correct, we will display a QMessageBox which tells the user that login is successful. Otherwise, another QMessageBox is displayed to alert the user to an error.

If the user's information does not exist, they can click the sign up button and a new window will appear where they can enter their information. This part is covered in the section "Project 3.2 – Create New User GUI."

Finally, we will learn how to change the event handler for when the user closes the window. Rather than just closing the application, we will first display a dialog box that will confirm whether or not the user really wants to quit.

The QPushButton Widget

Let's first take a look at a fundamental widget that you will probably use in almost every GUI you create, **QPushButton**. The QPushButton can be used to command the computer to perform some kind of operation or answer a question. When you click the QPushButton widget, it sends out a signal that can be connected to a function. Common buttons you might encounter are OK, Next, Cancel, Close, Yes, and No.

Buttons are typically displayed with either a text label or an icon that describes its action or purpose. There are a number of different kinds of buttons with different usages that can be created including `QToolButtons` and `QRadioButton`s.

For our first example, we are going to take a look at how to set up a `QPushButton` that, when clicked, will call a function that closes our application (Listing 3-1).

```
# button.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import QApplication, QWidget, QLabel, QPushButton

class ButtonWindow(QWidget):
    def __init__(self):
        super().__init__() # create default constructor for QWidget
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        self.setGeometry(100, 100, 200, 150)
        self.setWindowTitle('QPushButton Widget')
        self.displayButton() # call our displayButton function
        self.show()
    def displayButton(self):
        """
        Setup the button widget.
        """
        name_label = QLabel(self)
        name_label.setText("Don't push the button.")
        name_label.move(60, 30) # arrange label

        button = QPushButton('Push Me', self)
        button.clicked.connect(self.buttonClicked)
        button.move(80, 70) # arrange button
    def buttonClicked(self):
        """
        Print message to the terminal,
        and close the window when button is clicked.
        """
        print("The window has been closed.")
        self.close()

# Run program

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = ButtonWindow()
    sys.exit(app.exec_())
```

Listing 3-1 Code for learning how to add `QPushButton` widgets to your application

When you finish, your window should look similar to Figure 3-3.



Figure 3-3 Example of the QPushButton widget

Explanation

We begin by importing `sys` and the necessary PyQt classes including `QApplication` and `QWidget` for creating our application object and window, respectively. For this program we will also import the `QLabel` and `QPushButton` widgets which are also part of the `QtWidgets` module.

Next let's create our own `ButtonWindow` class which inherits from `QWidget`. Here we will initialize the window and widgets we need for our GUI. The `ButtonWindow` class has two functions, `displayButton` and `buttonClicked`. In the `displayButton` function, we create a label and set its text using `setText()`. If you look at the portion of code where the button is created, we set the text on the button as a parameter of `QPushButton`. We could also set the text as follows:

```
button.setText("Don't push the button.")
```

When you click the `QPushButton` with your mouse, it will send out the signal `clicked()`. After we create the button, use

```
button.clicked.connect(self.buttonClicked)
```

to connect the signal to the action we want the button to perform, in this case `self.buttonClicked`. A `QPushButton` widget can also be set to be activated by the spacebar or using a keyboard shortcut. The `buttonClicked` function calls `self.close()` to close the application.

Note In the preceding example, the signal `clicked()` is connected to our function. There are also other kinds of signals that the `QPushButton` can send out including `pressed()` when the button is down, `released()` when the button is released, or `toggled()` that can be used like a binary switch.

Events, Signals, and Slots

Before we go on, you should be introduced to an important concept when building GUI applications in PyQt. GUIs are **event-driven**, meaning that

they respond to events that are created by the user, from the keyboard or the mouse, or by events caused by the system, such as a timer or when connecting to Bluetooth. No matter how they are generated, the application needs to listen for these events and respond to them in some way, also known as **event handling**. For example, when `exec_()` is called, the application begins listening for events until it is closed.

In PyQt, event handling is performed with signals and slots. **Signals** are the events that occur when a widget's state changes, such as when a button is clicked or a checkbox is toggled on or off. Those signals then need to be handled in some way. **Slots** are the methods that are executed in response to the signal. Slots are simply Python functions or built-in PyQt functions that are connected to an event and executed when the signal occurs.

Take a look at the following code from the earlier `QPushButton` program:

```
button.clicked.connect(self.buttonClicked)
```

When we push on the button, a `clicked()` signal is emitted. In order to make use of that signal, we must `connect()` to some callable function, in this case `buttonClicked()`, which is the slot.

Put simply, widgets send out signals and we collect and use them with slots to make our application perform some action.

Many widgets have predefined signals and slots, meaning you only need to call them in order to get the behavior you want for your application.

The topic of signals and slots and how to make some custom signals will be covered in more detail and with examples in Chapter 6.

The QLineEdit Widget

The next widget we are going to take a look at is the **QLineEdit** widget. For our login GUI, we need to create areas where the user can input the text for their username and password on a single line. `QLineEdit` also supports normal text editing functions such as cut, copy and paste, and redo or undo if you need to add those features to your program.

The `QLineEdit` widget also has a number of methods to add more functionality to your GUI, such as hiding text when it is entered, using placeholder text, or even setting a limit on the length of the text that can be input.

In Listing 3-2 we will take a look at how to set up the `QLineEdit` widget, retrieve the text using the `text()` function, and see how to clear the text that the user inputs.

NoteIf you need multiple lines to enter text in, use `QTextEdit`.

```
# lineedit.py
```

```

# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget,
                             QLabel, QLineEdit, QPushButton)
from PyQt5.QtCore import Qt
class EntryWindow(QWidget): # Inherits QWidget
    def __init__(self): # Constructor
        super().__init__() # Initializer which calls constructor for QWidget
        self.initializeUI() # Call function used to set up window
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setGeometry(100, 100, 400, 200)
        self.setWindowTitle('QLineEdit Widget')
        self.displayWidgets()
        self.show()
    def displayWidgets(self):
        """
        Setup the QLineEdit and other widgets.
        """
        # Create name label and line edit widgets
        QLabel("Please enter your name below.", self).move(100, 10)
        name_label = QLabel("Name:", self)
        name_label.move(70, 50)
        self.name_entry = QLineEdit(self)
        self.name_entry.setAlignment(Qt.AlignLeft) # The default alignment is AlignLeft
        self.name_entry.move(130, 50)
        self.name_entry.resize(200, 20) # Change size of entry field
        self.clear_button = QPushButton('Clear', self)
        self.clear_button.clicked.connect(self.clearEntries)
        self.clear_button.move(160, 110)
    def clearEntries(self):
        """
        If button is pressed, clear the line edit input field.
        """
        sender = self.sender()
        if sender.text() == 'Clear':
            self.name_entry.clear()

# Run program

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = EntryWindow()
    sys.exit(app.exec_())

```

Listing 3-2 Code for learning how to add QLineEdit widgets to your application

Take a look at Figure [3-4](#) to get an idea of how your GUI should look.

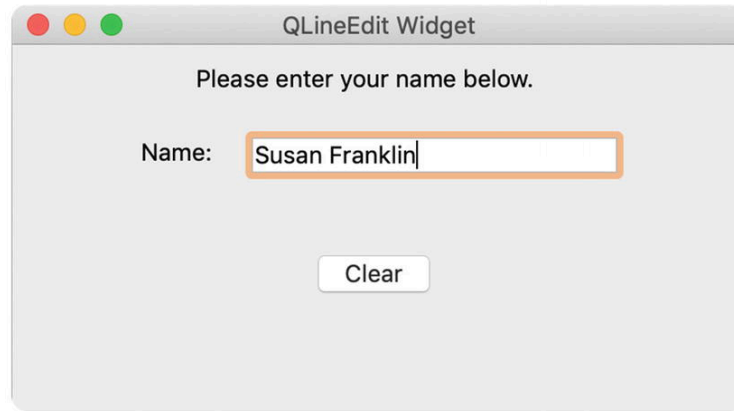


Figure 3-4 Example of how to use QLineEdit and QPushButton widgets

Explanation

The user can enter their name into the QLineEdit widget and click the “Clear” QPushButton to clear their text. Other features could include clearing multiple widgets’ states when the button is clicked or checking to make sure the text entered fits the guidelines you need in your application.

We begin by importing the necessary widgets, this time making sure to include the QLineEdit widget which is a member of the QtWidgets module. We also import Qt from the QtCore module. Qt contains various miscellaneous methods for creating GUIs. After initializing our window in the EntryWindow class, the displayWidgets() function is called that sets up the label, line edit and button widgets. When text is entered into the name_entry widget, by default, the text starts on the left and is centered vertically.

```
self.name_entry.setAlignment(Qt.AlignLeft)
```

If you wish to change this, you could change the flag in SetAlignment from Qt.AlignLeft to Qt.AlignRight or Qt.AlignHCenter.

When the clear_button is clicked, it emits a signal that is connected to the clearEntries() function. In order to determine where the source of a signal is coming from in your applications, you could also use the sender() method. Here, the signal is sent from our button when it is clicked, and if the text on the sender is 'Clear', then the name_entry widget reacts to the signal and clears its current text.

The QCheckBox Widget

The QCheckBox widget is a selectable button that generally has two states, on or off. Since checkboxes normally have only two states, they are perfect for representing features in your GUI that can either be enabled or disabled or for selecting from a list of options like in a survey.

The QCheckBox can also be used for more dynamic applications, as well. For example, you could use the checkbox to change the title of the window or even the text of labels when enabled.

Listing 3-3 shows how to set up a window like in a questionnaire. The user is allowed to select all checkboxes that apply to them, and each time the user clicks a box, we call a function to show how to determine the widget's current state.

NoteThe checkboxes in `QCheckBox` are not mutually exclusive, meaning you can select more than one checkbox at a time. To make them mutually exclusive, add the checkboxes to a `QButtonGroup` object.

```
# checkboxes.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QCheckBox, QLabel)
from PyQt5.QtCore import Qt

class CheckBoxWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        self.setGeometry(100, 100, 250, 250)
        self.setWindowTitle('QCheckBox Widget')
        self.displayCheckBoxes()
        self.show()
    def displayCheckBoxes(self):
        """
        Setup the checkboxes and other widgets
        """
        header_label = QLabel(self)
        header_label.setText("Which shifts can you work? (Please check all that apply)")
        header_label.setWordWrap(True)
        header_label.move(10, 10)
        header_label.resize(230, 60)
        # Set up checkboxes
        morning_cb = QCheckBox("Morning [8 AM-2 PM]", self) # text, parent
        morning_cb.move(20, 80)
        #morning_cb.toggle() # uncomment if you want box to start off checked,
        # shown as an example here.
        morning_cb.stateChanged.connect(self.printToTerminal)
        after_cb = QCheckBox("Afternoon [1 PM-8 PM]", self) # text, parent
        after_cb.move(20, 100)
        after_cb.stateChanged.connect(self.printToTerminal)
        night_cb = QCheckBox("Night [7 PM-3 AM]", self) # text, parent
        night_cb.move(20, 120)
        night_cb.stateChanged.connect(self.printToTerminal)
    def printToTerminal(self, state): # pass state of checkbox
        """
        Simple function to show how to determine the state of a checkbox.
        Prints the text label of the checkbox by determining which widget is sending the signal
        """
        sender = self.sender()
        if state == Qt.Checked:
            print("{} Selected.".format(sender.text()))
        else:
            print("{} Deselected.".format(sender.text()))
```

```
# Run program
if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = CheckBoxWindow()
    sys.exit(app.exec_())
```

Listing 3-3 Code for learning how to add QCheckBox widgets to your application

Figure 3-5 shows our application that allows users to select multiple checkboxes.

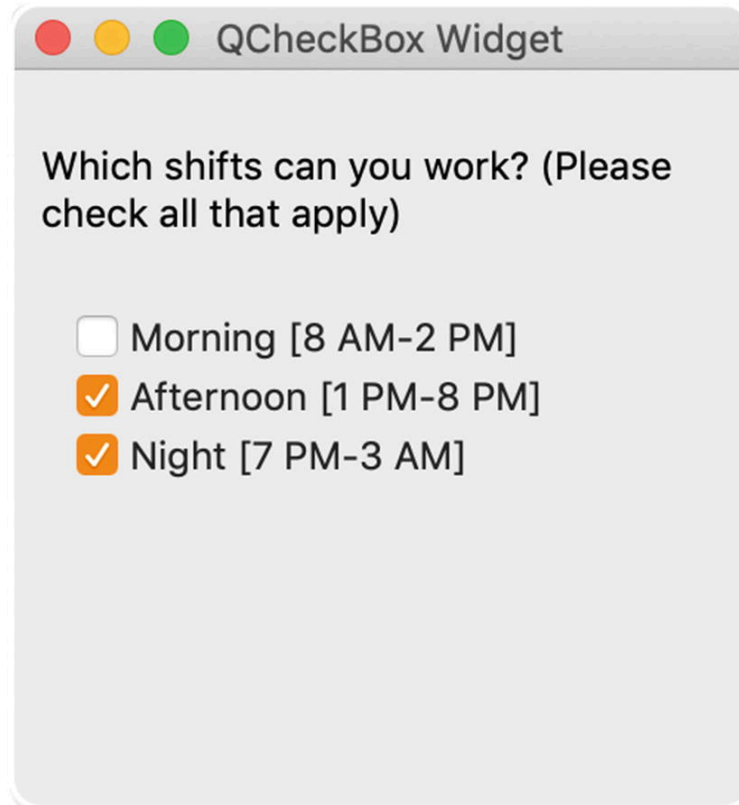


Figure 3-5 Example of QCheckBox widgets

Explanation

Much of this application starts off similar to before, so let's jump right into the `displayCheckBoxes()` method within the `CheckBoxWindow` class.

A `QLabel` widget is created so that the person looking at the window can understand the purpose of the GUI. Then three checkboxes are created, each with a variable name that is representative of the widget's purpose. Since the widgets are created in a similar manner, we will just take a look at the first one, `morning_cb`.

```
morning_cb = QCheckBox("Morning [8 AM-2 PM]", self)
morning_cb.move(20, 80) # arrange widget in window
#morning_cb.toggle() # uncomment if you want box to start off checked, shown as an example here
morning_cb.stateChanged.connect(self.printToTerminal)
```

The checkbox is created by calling the `QCheckBox` class and then, as parameters, assigning it text that will appear beside the actual checkbox and its parent window. The `toggle()` method can be used to toggle the checkbox, and uncommenting the code will cause the widget to begin as

enabled when starting the program. When a checkbox’s state changes, rather than using `clicked()` like with the `QPushButton`, we can use `stateChanged()` to send a signal and then `connect` to our function, `printToTerminal()`.

The `printToTerminal()` function takes as a parameter `state`, the state of the checkbox. If a checkbox is checked, we can find out by using the `isChecked()` method. If the state of the button is `isChecked()`, then use the `sender()` method to find out which button is sending a signal and print its text value to the terminal window. An example of the output to the terminal can be seen in Figure 3-6.

```
Morning [8 AM-2 PM] Selected.
Night [7 PM-3 AM] Selected.
Afternoon [1 PM-8 PM] Selected.
Night [7 PM-3 AM] Deselected.
Morning [8 AM-2 PM] Deselected.
Afternoon [1 PM-8 PM] Deselected.
Morning [8 AM-2 PM] Selected.
█
```

Figure 3-6 Output to terminal from `QCheckBox` example program

The `QMessageBox` Dialog Box

Often when a user is going to close an application, save their work, or an error occurs, a dialog box will pop up and display some sort of key information. The user can then interact with that dialog box, often by clicking a button to respond to the prompt.

The `QMessageBox` dialog box can not only be used to alert the user to a situation but also to allow them to decide how to handle the issue. For example, if you close a document you just modified, you might get a dialog box asking you to Save, Don’t Save, or Cancel.

There are four types of predefined `QMessageBox` widgets in PyQt. For more details, refer to Table 3-1.

Table 3-1 Four types of `QMessageBox` widgets in PyQt. Images from www.riverbankcomputing.com

QMessageBox Icons	Types	Details
	Question	Ask the user a question.
	Information	Display information during normal operations.
	Warning	Report noncritical errors.

QMessageBox Icons	Types	Details
	Critical	Report critical errors.

Windows vs. Dialogs

When creating a GUI application, you will more than likely come across the terms windows and dialogs. However, windows and dialogs are not the same. Using dialog boxes in an application can make it both easier for you to develop your GUI and for the user to better understand and navigate through your application.

The **window** generally consists of menus, a toolbar, and other kinds of widgets within it that can often act as the main interface in a GUI application.

A **dialog box** will appear when the user needs to be prompted for additional information in order to continue, often to gather input such as an image or a file. After that information is given, the dialog box is normally destroyed. Dialog boxes can also be used to display options or information while a user is working in the main window. Most kinds of dialog boxes will have a parent window that will be used to determine the position of the dialog with respect to its owner. This also means that communication occurs between the window and the dialog box and allows for updates in the main window.

There are two kinds of dialog boxes, the **modal** dialog box and the **modeless** dialog box. Modal dialogs block user interaction from the rest of the program until the dialog box is closed. Modeless dialogs allow the user to interact with both the dialog and the rest of the application.

How dialog boxes appear and are used can often be influenced by the operating system you use and the guidelines set by that OS.

How to Display a QMessageBox

The QMessageBox class produces a modal dialog box, and in Listing 3-4, we will take a look at how to use two of the predefined QMessageBox message types, Question and Information.

```
# dialogs.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QLabel,
                             QMessageBox, QLineEdit, QPushButton)
from PyQt5.QtGui import QFont
class DisplayMessageBox(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI() # Call our function used to set up window
    def initializeUI(self):
        """
```

```

Initialize the window and display its contents to the screen
"""

self.setGeometry(100, 100, 400, 200)
self.setWindowTitle('QMessageBox Example')
self.displayWidgets()
self.show()
def displayWidgets(self):
    """
    Set up the widgets

    """
    catalogue_label = QLabel("Author Catalogue", self)
    catalogue_label.move(20, 20)
    catalogue_label.setFont(QFont('Arial', 20))
    auth_label = QLabel("Enter the name of the author you are searching for:", self)
    auth_label.move(40, 60)
    # Create author label and line edit widgets
    author_name = QLabel("Name:", self)
    author_name.move(50, 90)
    self.auth_entry = QLineEdit(self)
    self.auth_entry.move(95, 90)
    self.auth_entry.resize(240, 20)
    self.auth_entry.setPlaceholderText("firstname lastname")
    # Create search button
    search_button = QPushButton("Search", self)
    search_button.move(125, 130)
    search_button.resize(150, 40)
    search_button.clicked.connect(self.displayMessageBox)
def displayMessageBox(self):
    """
    When button is clicked, search through catalogue of names.
    If name is found, display Author Found dialog.
    Otherwise, display Author Not Found dialog.
    """
    # Check if authors.txt exists
    try:
        with open("files/authors.txt", "r") as f:
            # read each line into a list
            authors = [line.rstrip('\n') for line in f]
    except FileNotFoundError:
        print("The file cannot be found.")
    # Check for name in list
    not_found_msg = QMessageBox() # create not_found_msg object to avoid causing a 'referenced befo

    if self.auth_entry.text() in authors:
        QMessageBox().information(self, "Author Found", "Author found in catalogue!", QMessageBox.Q
    else:
        not_found_msg = QMessageBox.question(self, "Author Not Found", "Author not found in catalog
    if not_found_msg == QMessageBox.No:
        print("Closing application.")
        self.close()
    else:
        pass
# Run program
if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = DisplayMessageBox()
    sys.exit(app.exec_())

```

Listing 3-4 Code for learning how to display QMessageBox dialogs

Figure 3-7 shows the GUI for the QMessageBox example.

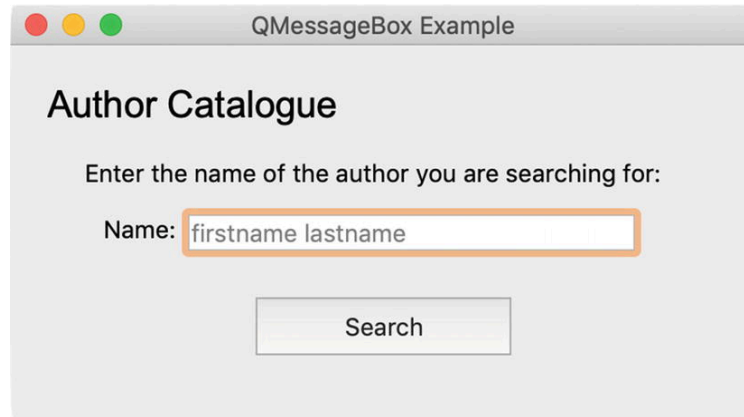


Figure 3-7 GUI to search for author's name in a text file

Explanation

The GUI in this example consists of a few `QLabel` widgets, a `QLineEdit` widget, and a single `QPushButton`. For this example, you will also see how to set placeholder text in the `QLineEdit` widget using `setPlaceholderText()`. This can be helpful for a number of reasons, maybe to make the look of the window less cluttered or to give the user extra information to help them understand the format to use to input text.

The `search_button` sends a signal that calls the function `displayMessageBox()`. If the user enters a name that is contained in the `authors.txt` file, then an information dialog box appears like the first image in Figure 3-8. Otherwise, a question dialog box (second image in Figure 3-8) appears asking the user if they want to search again or quit the program. Let's take a look at how to create a dialog box using the `QMessageBox` class.

```
not_found_msg = QMessageBox.question(self, "Author Not Found", "Author not found in catalogue.\nDo you
```

To create a `QMessageBox` dialog, we first call `QMessageBox` and choose one of the predefined types, in this case `question`. Then we set the dialog title, "Author Not Found", and the text that we want to appear inside the dialog. This should inform the user about the current situation and, if necessary, notify them of actions they could take. This is followed by the types of buttons that will appear in the dialog, and each button is separated by a pipe key, `|`. Other types of buttons include `Open`, `Save`, `Cancel`, and `Reset`. Finally, you can specify which button you want to highlight and set as the default button.

Note On Mac OS X, when a message box appears, the title is generally ignored due to Mac OS X Guidelines. If you are using a Mac and don't see a title in the dialog boxes, don't fear! You haven't done anything wrong.

You can also specify each of these fields in separate lines by calling `setText()`, `setWindowTitle()`, and other methods.

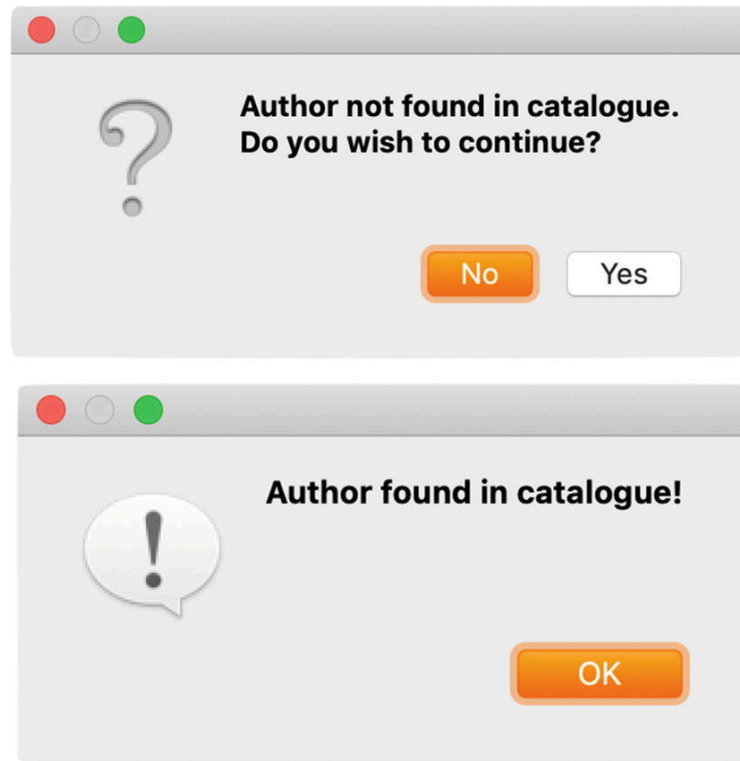


Figure 3-8 Information dialog box (top) that lets the user know that their search was successful. However, if the author doesn't exist, a question dialog box appears asking the user to take some sort of action by clicking a button (bottom)

Login GUI Solution

Now that we have covered the key widgets in this chapter and how to implement dialog boxes, we should have all the necessary concepts down to tackle the login GUI (Listing 3-5). Refer to Figures 3-1 and 3-2 for the look and layout of the login GUI.

```
# loginUI.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QLabel, QMessageBox, QLineEdit, QPushButton, QCheckBox)
from PyQt5.QtGui import QFont
from PyQt5.QtCore import Qt
from Registration import CreateNewUser # Import the registration module

class LoginUI(QWidget):
    def __init__(self): # Constructor
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setGeometry(100, 100, 400, 230)
        self.setWindowTitle('3.1 - Login GUI')
        self.loginUserInterface()
        self.show()
    def loginUserInterface(self):
        """
        Create the login GUI.
        """
        login_label = QLabel(self)
        login_label.setText("login")
```

```

login_label.move(180, 10)
login_label.setFont(QFont('Arial', 20))
# Username and password labels and line edit widgets
name_label = QLabel("username:", self)
name_label.move(30, 60)
self.name_entry = QLineEdit(self)
self.name_entry.move(110, 60)
self.name_entry.resize(220, 20)
password_label = QLabel("password:", self)
password_label.move(30, 90)
self.password_entry = QLineEdit(self)
self.password_entry.move(110, 90)

self.password_entry.resize(220, 20)
# Sign in push button
sign_in_button = QPushButton('login', self)
sign_in_button.move(100, 140)
sign_in_button.resize(200, 40)
sign_in_button.clicked.connect(self.clickLogin)
# Display show password checkbox
show_pswd_cb = QCheckBox("show password", self)
show_pswd_cb.move(110, 115)
show_pswd_cb.stateChanged.connect(self.showPassword)
show_pswd_cb.toggle()
show_pswd_cb.setChecked(False)
# Display sign up label and push button
not_a_member = QLabel("not a member?", self)
not_a_member.move(70, 200)
sign_up = QPushButton("sign up", self)
sign_up.move(160, 195)
sign_up.clicked.connect(self.createNewUser)
def clickLogin(self):
    """
    When user clicks sign in button, check if username and password match any existing profiles in
    If they exist, display messagebox and close program.
    If they don't, display error messagebox.
    """
    users = {} # Create empty dictionary to store user information
    # Check if users.txt exists, otherwise create new file
    try:
        with open("files/users.txt", 'r') as f:
            for line in f:
                user_fields = line.split(" ")
                username = user_fields[0]
                password = user_fields[1].strip('\n')
                users[username] = password
    except FileNotFoundError:
        print("The file does not exist. Creating a new file.")
        f = open("files/users.txt", "w")
        username = self.name_entry.text()

        password = self.password_entry.text()
        if (username, password) in users.items():
            QMessageBox.information(self, "Login Successful!", "Login Successful!", QMessageBox.Ok,
                                   self.close() # close program
            else:
                QMessageBox.warning(self, "Error Message", "The username or password is incorrect.", QMessa
def showPassword(self, state):
    ...

```

```

        If checkbox is enabled, view password.
        Else, mask password so others cannot see it.
        ...

        if state == Qt.Checked:
            self.password_entry.setEchoMode(QLineEdit.Normal)
        else:
            self.password_entry.setEchoMode(QLineEdit.Password)
    def createNewUser(self):
        """
        When the sign up button is clicked, open
        a new window and allow the user to create a new account.
        """
        self.create_new_user_dialog = CreateNewUser()
        self.create_new_user_dialog.show()
    def closeEvent(self, event):
        """
        Display a QMessageBox when asking the user if they want to quit the program.
        """
        # Set up message box
        quit_msg = QMessageBox.question(self, "Quit Application?",
            "Are you sure you want to Quit?", QMessageBox.No | QMessageBox.Yes,
            QMessageBox.Yes)
        if quit_msg == QMessageBox.Yes:
            event.accept() # accept the event and close the application
        else:
            event.ignore() # ignore the close event
# Run program

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = LoginUI()
    sys.exit(app.exec_())

```

Listing 3-5 Code for login GUI

Your GUI should look similar to the window shown in Figure 3-1.

Explanation

After importing the necessary PyQt5 modules including `QtWidgets`, `QtGui`, and `QtCore`, we also need to import our `Registration` module which will allow new users to create a new account and then return back to the login GUI to sign in. The `Registration` module is covered in Project 3.2 in this chapter.

In our `LoginUI` class that inherits from `QWidget`, we initialize our GUI and then create the widgets. Refer to Figure 3-2 for the layout. We create a few `QLabel` widgets to hold information about our GUI and labels for the `QLineEdit` widgets for the username and password. Widgets are arranged in the window using the `move()` method.

When the `sign_in_button` is clicked, it sends a signal that is connected to the `clickLogin()` method. This function opens the `users.txt` file (and creates one if it doesn't exist) and stores each line into a Python dictionary, with the keys being the usernames and the values of the dictionary being the passwords. The `text()` method is then used to retrieve the input from the two `QLineEdit` widgets and checks them to see if they match

any of the key/value pairs in the `users` dictionary. While it isn't the most practical method, this example is a very small one and demonstrates how to use simple text files with your applications. Later we will take a look at how to use SQL to search through databases in Chapter [10](#).

If the username and password match a key/value pair from the file, then an information `QMessageBox` dialog is displayed telling the user that they are successful. They can then exit the program if they wish (in an actual application at this point you would start the main window of your program). Otherwise, a warning `QMessageBox` is displayed if the username or password is incorrect. These two dialog boxes can be seen in Figure [3-9](#).

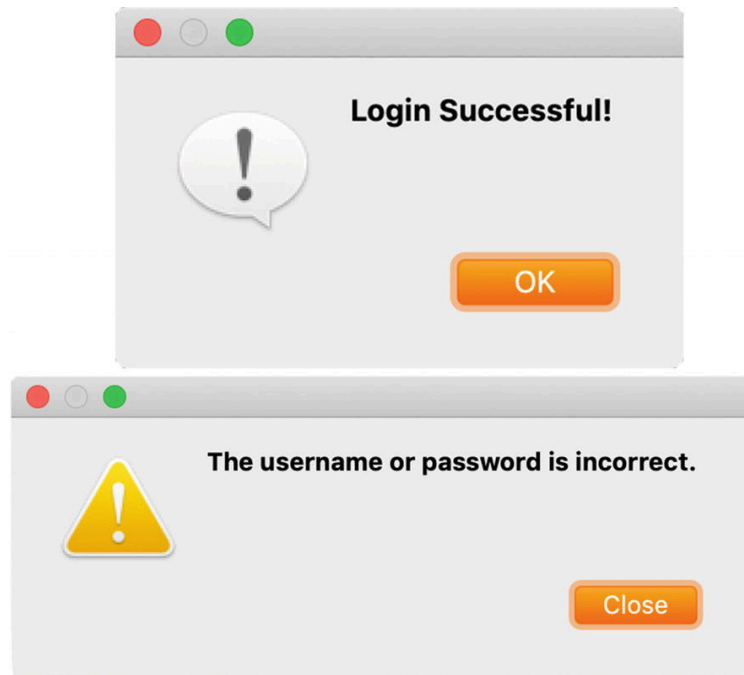


Figure 3-9 `QMessageBox` dialogs that can be displayed. The information dialog box (top) lets the user know that their information was correct. The other dialog (bottom) shows a warning `QMessageBox`

Hiding Input for `QLineEdit`

The `stateChanged` signal in the login UI code is connected to the `showPassword()` function. If the `show_pswd_cb` `QCheckBox` is checked, then the password is displayed using `setEchoMode()`.

```
self.password_entry.setEchoMode(QLineEdit.Normal)
```

Otherwise, if unchecked, it is hidden using

```
self.password_entry.setEchoMode(QLineEdit.Password)
```

If you ever need to make the text in a `QLineEdit` widget hidden from other's view, using `setEchoMode()` can change the appearance of the text. By default, `setEchoMode()` is set to `QLineEdit.Normal`.

How to Open a New Window

If the user wants to create a new account, then they can click the `sign_up` button at the bottom of the GUI. This button sends a signal that is connected to the `createNewUser()` method which calls our `CreateNewUser` class from the `Registration` module. A new window is then opened up using `show()` where the user can enter their personal information.

Changing How the Close Event Works

Finally, currently when we want to quit our programs, we just exit by clicking the button in the top corner of the window. However, a good practice is to present a dialog box confirming whether the user really wants to quit or not. In most programs this will prevent the user from forgetting to save their latest work.

When a `QWidget` is closed in PyQt, it generates a `QCloseEvent`. So we need to change how the `closeEvent()` method is handled. To do so we create a new method called `closeEvent()` that accepts as a parameter an event.

In this function we create a `QMessageBox` that asks the user if they are sure about quitting. They then can click either a Yes or No button in the dialog box. We then check the value of the variable stored in `quit_msg`. If `quit_msg` is Yes, then the close event is accepted and the program is closed. Otherwise, the event is ignored.

Project 3.2 – Create New User GUI

The first time someone uses your applications you may want them to sign up and create their own username and passwords. This, of course, can allow them to personalize their accounts and then save that information for the next time they log in. The kind of information that you need from the user can range from very simple, name and gender, all the way to extremely private, Social Security numbers or bank account information. Making sure that the information that they enter is correct is very important.

Creating a New User GUI Solution

For the following project, we will have the user enter their desired username, their real name, a password, and then reenter that password to double check that it is correct. When the user clicks the sign up button, the text in the password fields will be checked for a match, and if so, the information will be saved to a text file. The user can then return to the login screen and log in.

The create new user GUI project contains many of the same widgets, including `QLabel` widgets, `QLineEdit` widgets, a `QPushButton`, and concepts that were part of the login UI project. Therefore, we will jump right into talking about the code shown in Listing 3-6. The completed GUI can be seen in Figure 3-10.

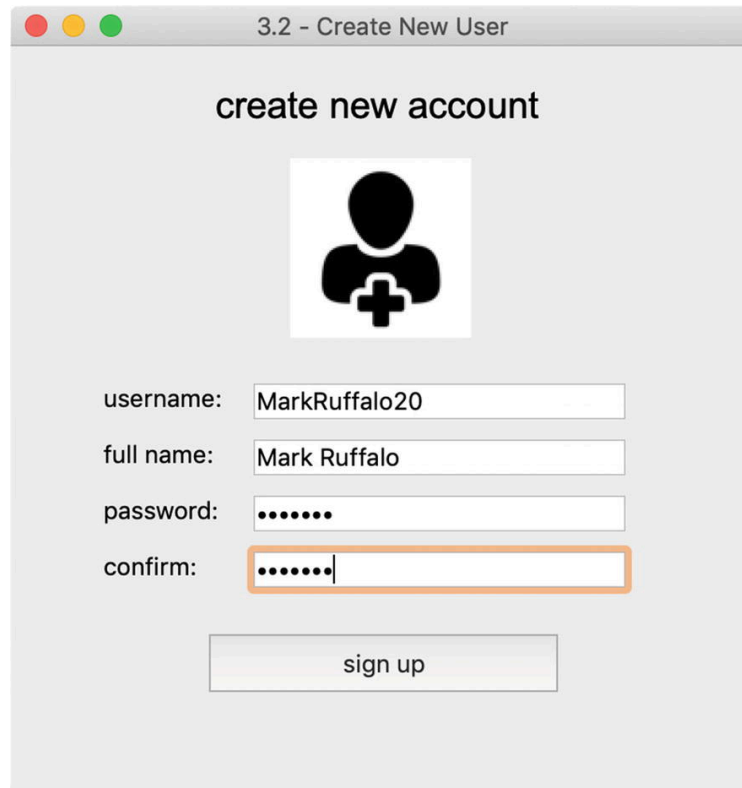


Figure 3-10 The create new user GUI

```
# Registration.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QMessageBox, QPushButton, QLabel, QLineEdit)
from PyQt5.QtGui import QFont, QPixmap
class CreateNewUser(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI() # Call our function used to set up window
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setGeometry(100, 100, 400, 400)
        self.setWindowTitle('3.2 - Create New User')
        self.displayWidgetsToCollectInfo()
        self.show()
    def displayWidgetsToCollectInfo(self):
        """
        Create widgets that will be used to collect information
        from the user to create a new account.
        """
        # Create label for image
        new_user_image = "images/new_user_icon.png"
        try:
            with open(new_user_image):
                new_user = QLabel(self)

                pixmap = QPixmap(new_user_image)
                new_user.setPixmap(pixmap)
                new_user.move(150, 60)
        except FileNotFoundError:
```

```

        print("Image not found.")
    login_label = QLabel(self)
    login_label.setText("create new account")
    login_label.move(110, 20)
    login_label.setFont(QFont('Arial', 20))
    # Username and fullname labels and line edit widgets
    name_label = QLabel("username:", self)
    name_label.move(50, 180)
    self.name_entry = QLineEdit(self)
    self.name_entry.move(130, 180)
    self.name_entry.resize(200, 20)
    name_label = QLabel("full name:", self)
    name_label.move(50, 210)
    name_entry = QLineEdit(self)
    name_entry.move(130, 210)
    name_entry.resize(200, 20)
    # Create password and confirm password labels and line edit widgets
    pswd_label = QLabel("password:", self)
    pswd_label.move(50, 240)
    self.pswd_entry = QLineEdit(self)
    self.pswd_entry.setEchoMode(QLineEdit.Password)
    self.pswd_entry.move(130, 240)
    self.pswd_entry.resize(200, 20)
    confirm_label = QLabel("confirm:", self)
    confirm_label.move(50, 270)
    self.confirm_entry = QLineEdit(self)
    self.confirm_entry.setEchoMode(QLineEdit.Password)
    self.confirm_entry.move(130, 270)
    self.confirm_entry.resize(200, 20)
    # Create sign up button
    sign_up_button = QPushButton("sign up", self)
    sign_up_button.move(100, 310)
    sign_up_button.resize(200, 40)
    sign_up_button.clicked.connect(self.confirmSignUp)
def confirmSignUp(self):
    """
    When user presses sign up, check if the passwords match

    If they match, then save username and password text to users.txt.
    """
    pswd_text = self.pswd_entry.text()
    confirm_text = self.confirm_entry.text()
    if pswd_text != confirm_text:
        # Display messagebox if passwords don't match
        QMessageBox.warning(self, "Error Message",
            "The passwords you entered do not match. Please try again.", QMessageBox.Close,
            QMessageBox.Close)
    else:
        # If passwords match, save passwords to file and return to login
        # and test if you can log in with new user information.
        with open("files/users.txt", 'a+') as f:
            f.write(self.name_entry.text() + " ")
            f.write(pswd_text + "\n")
        self.close()
# Run program
if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = CreateNewUser()

```

```
sys.exit(app.exec_())
```

Listing 3-6 Code for creating a new user account GUI

The completed create new user GUI can be seen in Figure 3-10.

Explanation

The create new user GUI is mainly comprised of a few `QLabel` widgets, four `QLineEdit` widgets, and a `QPushButton` widget that the user can click when the form is complete as can be seen in Figure 3-10.

After the user enters their information and clicks the `sign_up_button`, the `confirmSignUp()` function is called and first checks to see if the text in the `pswd_entry` and `confirm_entry` `QLineEdit` objects match. If they don't match, then a `QMessageBox` like the one in Figure 3-11 is displayed. Otherwise, the text in the `name_entry` and `pswd_entry` fields is saved to a newline in the `users.txt` file separated by a space which can be seen in Figure 3-12.



Figure 3-11 The warning dialog displayed if the passwords you entered don't match

If the user finishes the form and clicks the `sign_up_button` or closes the window before completing the form, the window will close but the login UI will still remain open. If the form was completed, the user can try to enter their new username and password into the login GUI to log in.

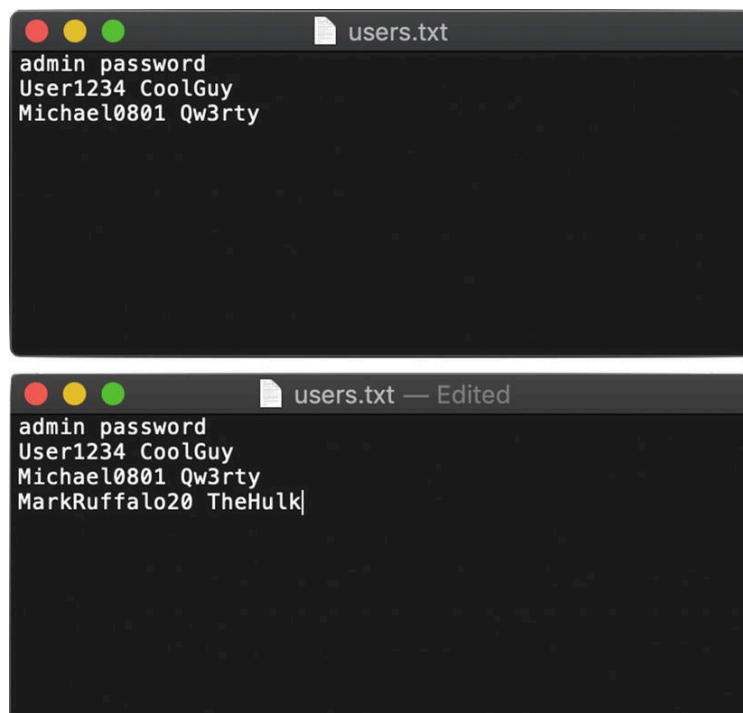


Figure 3-12 The original users.txt file (top) and the updated one with a new username and a new password (bottom)

Summary

In this chapter we took a look at some new widgets, QPushButton, QLineEdit, QCheckBox, and QMessageBox class. It is important to use dialog boxes in your program when you want the user to collect information outside of the application or to relay important details to the user. But you also should not have a dialog box pop up for every little nuance or with very little helpful information.

The applications in this chapter are by no means complete. They are the framework to get you started making your own GUIs. For example, you could make sure that the user's password includes capital and lowercase letters and other characters to make it more safe. Another possibility is to let the user know if the username they want to create already exists. Once you learn how to implement menus, you could even have the user search through their files for a profile image. I encourage you to try and implement some of these ideas or even your own ideas.

In the following chapter, we will learn about layout management in PyQt.